Avance Proyecto 2:
Pipeline IoT/Edge

IL355 - Programación de Sistemas Avanzados

Backlog

Infraestructura

- Crear estructura de carpetas
- Configurar `.gitignore`
- Crear `docker-compose.yml`
- Crear `README.md` inicial
- Documentar red (VPN)

Sistema en Rust

- Crear workspace de Rust
- Módulos: sensor, edge, coord
- Definir mensajes (JSON)
- Sensor genera datos
- Edge procesa/reenvía
- Coordinator recibe métricas
- Implementar heartbeats

Simulación Red

- Script baseline (limpieza)
- Script de latencia (netem)
- Probar con iperf3 y doc.

Metodología y Gestión del Proyecto

Estrategia Ágil

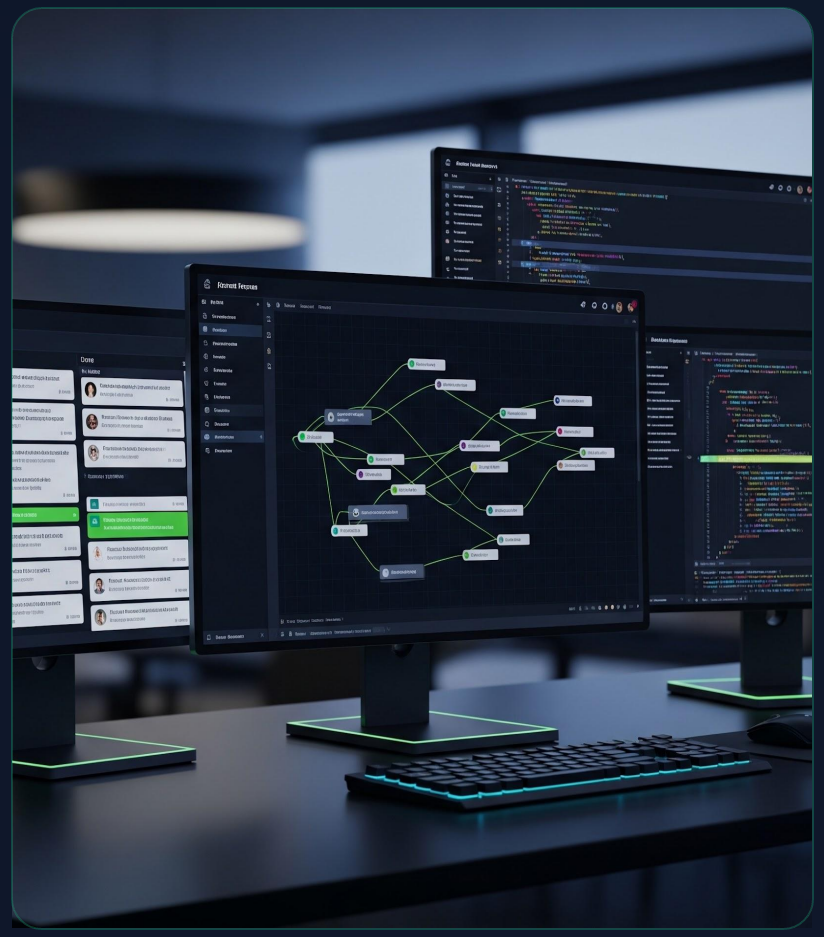
Se implementa una adaptación de Scrum para gestionar la complejidad técnica del monitor de red.

- Sprints: Duración de 2 semana con entregas incrementales.
- Roles: Líder de Proyecto, encargado de Documentación, y Desarrolladores.
- Revisiones: Sincronización diaria para resolver bloqueos.

Herramientas

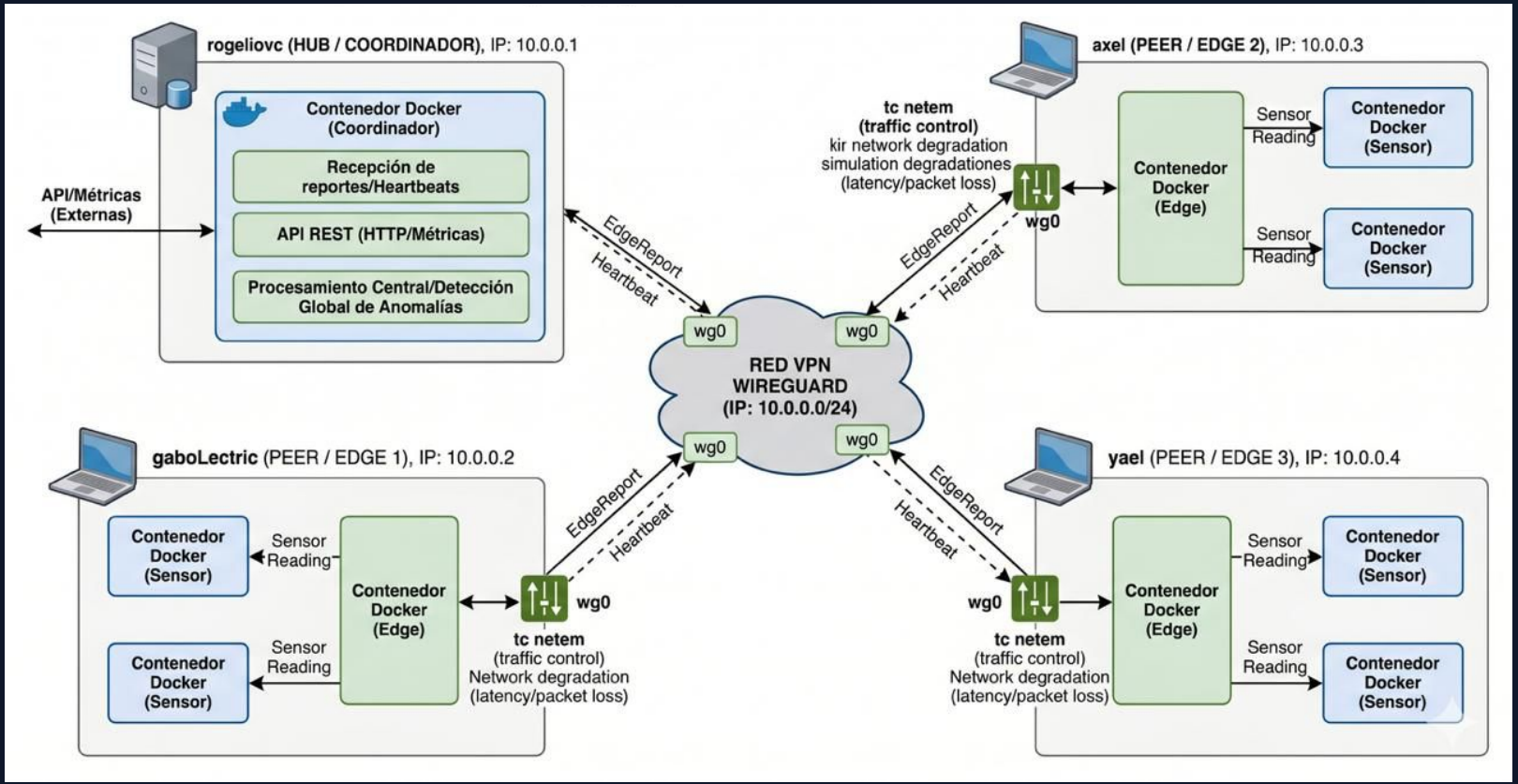
Ecosistema colaborativo para asegurar la trazabilidad y calidad del software.

- Gestión: Github Projects para el seguimiento del Backlog con issues y PRs.
- Código:
- Comunicación: Meet/WhatsApp para validación ágil y técnica.



Topología de Red, VPN, Dockers

Implementación con WireGuard



Nivel de VPN Declarado: [1] (Sin CGNAT, IP pública disponible).

El nodo central (Hub), administrado por Rogelio (rogeliovc), aloja el contenedor del Coordinador y cuenta con una conexión a Internet provista de una dirección IP pública directa, libre de restricciones de Carrier-Grade NAT (CGNAT), por lo que se usó PORT-FORWARDING directamente en el router del domicilio de rogeliovc.

Arquitectura del Pipeline

1. Sensor

Módulo encargado de generar datos simulados. Se ejecuta dentro de su propio contenedor y envía su flujo de información hacia el nodo intermedio (Edge).

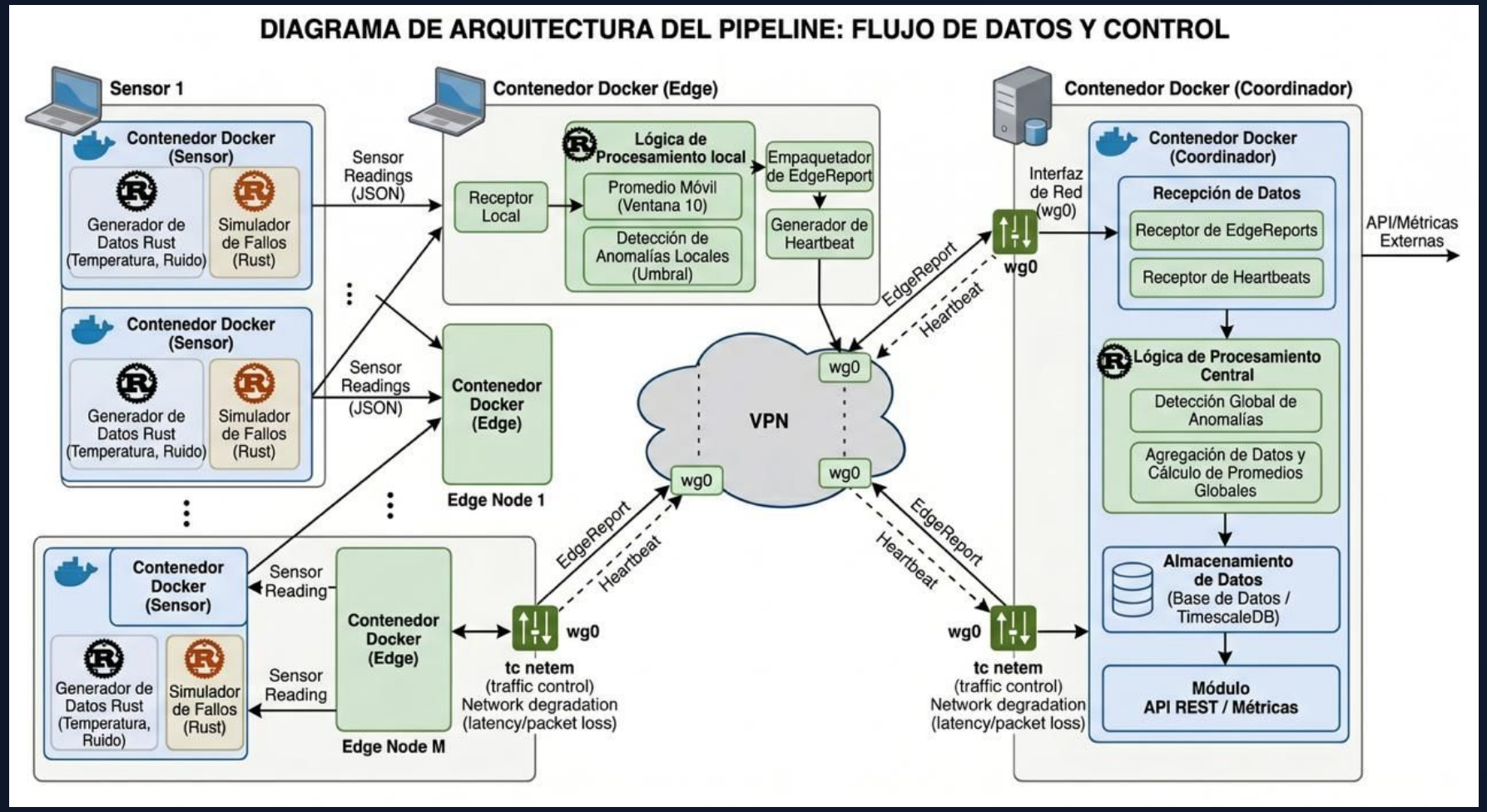
2. Edge Node

Contenedor intermedio crítico. Recibe, procesa y enruta los datos (ej. procesamiento de Mandelbrot). Diseñado con principios de tolerancia a fallos.

3. Coordinator

Nodo central responsable de recolectar y consolidar la información proveniente de múltiples nodos Edge en la red distribuida.

Arquitectura del pipeline



Esta topología asegura que todo el tráfico entre Sensor, Edge y Coordinator viaja encapsulado, permitiendo la inyección de latencia de red realista a nivel de host sin aislar los contenedores Docker.

Prototipo en Rust

El código base ha sido estructurado en espacios de trabajo (workspaces) usando Cargo, promoviendo la modularidad y reutilización:

- **rust/common/:** Definiciones de estructuras de datos compartidas y utilidades de red (`lib.rs`).
- **rust/sensor/:** Binario independiente que actúa como cliente inyector de datos.
- **rust/edge/:** Procesamiento intermedio. Mantiene el estado de conexión para reintentos en caso de fallo.
- **rust/coordinator/:** Servidor principal (`main.rs`) que escucha las conexiones entrantes y centraliza logs.

rust > Cargo.toml

```toml
[workspace]
members = [
    "common",
    "sensor",
    "edge",
    "coordinator"
]
resolver = "2"
```

Prototipo en Rust

Coordinator

main.rs

rust > coordinator > src > main.rs

```rust
use axum::{
    routing::{get, post},
    Json, Router, extract::State,
};
use common::{EdgeReport, Heartbeat, CoordStatus};
use std::collections::HashMap;
use std::sync::{Arc, Mutex, MutexGuard};
use std::time::{Duration, SystemTime, UNIX_EPOCH};
use tokio::time;

// Colores ANSI por Edge (para distinguir en capturas de pantalla)
const EDGE_COLORS: [&str; 10] = [
    "\x1b[36m", // Cyan
    "\x1b[33m", // Amarillo
    "\x1b[35m", // Magenta
    "\x1b[32m", // Verde
    "\x1b[34m", // Azul
    "\x1b[91m", // Rojo claro
    "\x1b[96m", // Cyan claro
    "\x1b[93m", // Amarillo claro
    "\x1b[95m", // Magenta claro
    "\x1b[92m", // Verde claro
];
const NC: &str = "\x1b[0m"; // Reset

fn edge_color(edge_id: &str) -> &'static str {
    let mut hash: u64 = 0;
    for b in edge_id.bytes() {
        hash = hash.wrapping_mul(31).wrapping_add(b as u64);
    }
    EDGE_COLORS[(hash % EDGE_COLORS.len() as u64) as usize]
}

/// Helper: obtener lock tolerante a poison (si un hilo paniquea, el Mutex
/// queda "envenenado"; con esto recuperamos el estado en vez de crashear).
fn safe_lock<T>(mutex: &Mutex<T>) -> MutexGuard<'_, T> {
    mutex.lock().unwrap_or_else(|poisoned| {
        eprintln!("[WARN] Mutex envenenado — recuperando estado");
        poisoned.into_inner()
    })
}

// Estructura interna para rastrear la salud de cada Edge
#[derive(Debug, Clone)]
struct NodeHealth {
    last_seen_ms: u64,
    is_online: bool,
    expected_seq: u64, // Para rastrear el siguiente mensaje esperado
}
```

Edge

main.rs

rust > edge > src > main.rs

```rust
use axum::{
    routing::post,
    Json, Router, extract::State,
};
use common::{SensorReading, EdgeReport};
use std::sync::{Arc, Mutex};
use std::time::{SystemTime, UNIX_EPOCH, Duration};
use tokio::time;
use std::sync::atomic::{AtomicU64, Ordering};

// Este es el estado compartido en memoria.
// Axum guarda las lecturas aquí, y la tarea en segundo plano las toma de aquí.
struct AppState {
    readings: Arc<Mutex<Vec<SensorReading>>>,
    edge_id: String,
    coordinator_url: String,
    coord_hb_url: String,
    sequence_counter: Arc<AtomicU64>,
}

#[tokio::main]
async fn main() {
    // Configuración con variables de entorno para Docker
    let edge_id = std::env::var("EDGE_ID").expect("FALTA VARIABLE: EDGE_ID");
    let coordinator_url = std::env::var("COORDINATOR_URL").expect("FALTA VARIABLE: COORDINATOR_URL");
    let coord_hb_url = std::env::var("COORD_HB_URL").expect("FALTA VARIABLE: COORD_HB_URL");
    let port = std::env::var("PORT").unwrap_or_else(|_| "4000".to_string());

    // Inicializamos el estado vacío
    let shared_state = Arc::new(AppState {
        readings: Arc::new(Mutex::new(Vec::new())),
        edge_id: edge_id.clone(),
        coordinator_url: coordinator_url.clone(),
        coord_hb_url: coord_hb_url.clone(),
        sequence_counter: Arc::new(AtomicU64::new(0)),
    });

    // Clonamos el puntero del estado para dárselo al hilo en segundo plano
    let bg_state = shared_state.clone();

```

Prototipo en Rust

Sensor

main.rs

rust > sensor > src > main.rs

```rust
use common::SensorReading;
use std::time::{SystemTime, UNIX_EPOCH, Duration};
use rand::Rng;
use tokio::time;

#[tokio::main]
async fn main() {
    // Usamos variables de entorno para que Docker Compose pueda inyectar las IPs reales
    let sensor_id = std::env::var("SENSOR_ID").expect("FALTA VARIABLE: SENSOR_ID");
    let edge_url = std::env::var("EDGE_URL").expect("FALTA VARIABLE: EDGE_URL");
    let coord_hb_url = std::env::var("COORD_HB_URL").expect("FALTA VARIABLE: COORD_HB_URL");

    println!("🚀 Iniciando {}...", sensor_id);
    println!("📡 Enviando ráfagas de datos a {}", edge_url);
    println!("❤️ Heartbeat configurado hacia {}", coord_hb_url);

    let client = reqwest::Client::new();

    // Hilo de Heartbeat para el Sensor
    let sensor_id_hb = sensor_id.clone();
    tokio::spawn(async move {
        let client = reqwest::Client::new();
        let mut interval = time::interval(Duration::from_secs(2));
        loop {
            interval.tick().await;
            let hb = common::Heartbeat {
                node_id: sensor_id_hb.clone(),
                role: "sensor".to_string(),
                timestamp_ms: SystemTime::now().duration_since(UNIX_EPOCH).unwrap().as_millis() as u64,
            };
            let _ = client.post(&coord_hb_url).json(&hb).send().await;
        }
    });

    // Configuramos el sensor para enviar 2 lecturas por segundo (cada 500ms)
    let mut interval = time::interval(Duration::from_millis(500));
    let mut rng = rand::thread_rng();

    loop {
        interval.tick().await;

        // Temperatura base aleatoria (ej. 20.0 a 30.0 grados)
        let base_temp: f64 = rng.gen_range(20.0..30.0);

        // Simular una anomalía térmica (> 80.0) con un 5% de probabilidad
        let temp = if rng.gen_bool(0.05) {
            rng.gen_range(85.0..95.0)
        } else {
            base_temp
        };
```

Elementos de la Demostración

Estado de la VPN Comprobación en vivo del estado del túnel usando comandos como wg show para confirmar el handshake.

Estado de Contenedores Ejecución de docker ps para verificar la salud y presencia de los tres tipos de contenedores orquestados.

Flujo en Tiempo Real Visualización de logs del *coordinator* y *edge nodes* para demostrar la correcta recepción y ruteo de datos.

Escenario con tc netem Aplicación en vivo de un escenario de degradación de red para demostrar el impacto en el pipeline.

Elementos de la Demostración

```
[rogeliovm@archlinux Pipeline_IoT-Edge]$ sudo docker ps
[sudo] password for rogeliovm:
CONTAINER ID   IMAGE                COMMAND           CREATED        STATUS         PORTS                                          NAMES
cd05b5de8f7c   docker-coordinator   "entrypoint.sh"   2 minutes ago  Up 2 minutes   0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp    coordinator
c2ab7636b8e0   docker-edge          "entrypoint.sh"   9 minutes ago  Up 9 minutes   0.0.0.0:4000->4000/tcp, [::]:4000->4000/tcp    edge-processor
f7c8674ee4bc   docker-sensor-1      "entrypoint.sh"   9 minutes ago  Up 9 minutes                                                  iot-sensor-1
1d9697410033   docker-sensor-2      "entrypoint.sh"   9 minutes ago  Up 9 minutes                                                  iot-sensor-2
[rogeliovm@archlinux Pipeline_IoT-Edge]$
```

```
gabolectric@ubuntu-dev:/$ ping -c 4 10.10.10.1
PING 10.10.10.1 (10.10.10.1) 56(84) bytes of data.
64 bytes from 10.10.10.1: icmp_seq=1 ttl=64 time=31.9 ms
64 bytes from 10.10.10.1: icmp_seq=2 ttl=64 time=343 ms
64 bytes from 10.10.10.1: icmp_seq=3 ttl=64 time=262 ms
^C
--- 10.10.10.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2007ms
rtt min/avg/max/mdev = 31.949/212.455/342.959/131.800 ms
gabolectric@ubuntu-dev:/$ sudo wg show
interface: wg0
  public key: 3aMW6AVbVPLAP0SWcdFpQF2C9jCC1pK8Hp9MzCQltR8=
  private key: (hidden)
  listening port: 51820

peer: NK3tGMGpPvjhFjbRNEGtinhMn/lEFp5V0roL4N+iNBw=
  endpoint: 187.212.178.33:51820
  allowed ips: 10.10.10.0/24
  latest handshake: 44 seconds ago
  transfer: 988 B received, 3.14 KiB sent
  persistent keepalive: every 25 seconds
gabolectric@ubuntu-dev:/$
```

```
coordinator  | [REPORTE] edge-gaboLectric | Muestras: 10 | Promedio: 24.1°C | Latencia red: 268ms | Seq: 468 | Edges activos: 3
coordinator  | [HEARTBEAT] edge-gaboLectric | Latencia red: 190ms | Rol: edge | Estado: ACTIVO
coordinator  | [HEARTBEAT] edge-rogelio | Latencia red: 0ms | Rol: edge | Estado: ACTIVO
coordinator  | [REPORTE] edge-rogelio | Muestras: 10 | Promedio: 25.7°C | Latencia red: 0ms | Seq: 29 | Edges activos: 3
coordinator  | [HEARTBEAT] edge-axel | Latencia red: 2052ms | Rol: edge | Estado: ACTIVO
coordinator  | [REPORTE] edge-axel | Muestras: 10 | Promedio: 31.4°C | Latencia red: 1058ms | Seq: 311 | Edges activos: 3
coordinator  | [HEARTBEAT] edge-gaboLectric | Latencia red: 157ms | Rol: edge | Estado: ACTIVO
coordinator  | [REPORTE] edge-gaboLectric | Muestras: 10 | Promedio: 31.4°C | Latencia red: 157ms | Seq: 469 | Edges activos: 3
coordinator  | [REPORTE] edge-rogelio | Muestras: 10 | Promedio: 25.1°C | Latencia red: 1ms | Seq: 30 | Edges activos: 3
coordinator  | [HEARTBEAT] edge-rogelio | Latencia red: 1ms | Rol: edge | Estado: ACTIVO
coordinator  | [HEARTBEAT] edge-axel | Latencia red: 110ms | Rol: edge | Estado: ACTIVO
coordinator  | [REPORTE] edge-axel | Muestras: 10 | Promedio: 29.9°C | Latencia red: 111ms | Seq: 312 | Edges activos: 3
coordinator  | [REPORTE] edge-rogelio | Muestras: 10 | Promedio: 39.8°C | Latencia red: 2ms | Seq: 31 | Edges activos: 3
coordinator  | [REPORTE] edge-gaboLectric | Muestras: 10 | Promedio: 30.5°C | Latencia red: 309ms | Seq: 470 | Edges activos: 3
coordinator  | [REPORTE] edge-axel | Muestras: 10 | Promedio: 24.0°C | Latencia red: 57ms | Seq: 313 | Edges activos: 3
```

Elementos de la Demostración

gabolectric — gabolectric@ubuntu-dev: ~/Pipeline_IoT-Edge — ssh ◂ boot_ubuntu.sh — 120×35

```
edge-processor | [Edge] Procesadas 10 lecturas. Latencia prom: 1405ms. Enviando...
edge-processor | [Edge] Reintentando en 100ms...
edge-processor | [Edge] Fallo al contactar al coordinador (intento 1): error sending request for url (http://10.10.10.1:
3000/submit_report): error trying to connect: tcp connect error: Connection refused (os error 111)
edge-processor | [Edge] Reintentando en 200ms...
edge-processor | [Edge] Fallo al contactar al coordinador (intento 2): error sending request for url (http://10.10.10.1:
3000/submit_report): error trying to connect: tcp connect error: Connection refused (os error 111)
edge-processor | [Edge] Reintentando en 400ms...
edge-processor | [Edge] Fallo al contactar al coordinador (intento 3): error sending request for url (http://10.10.10.1:
3000/submit_report): error trying to connect: tcp connect error: Connection refused (os error 111)
edge-processor | [Edge] Reintentando en 800ms...
edge-processor | [Edge] Fallo al contactar al coordinador (intento 4): error sending request for url (http://10.10.10.1:
3000/submit_report): error trying to connect: tcp connect error: Connection refused (os error 111)
edge-processor | [Edge] Ventana procesada: promedio 43.57, muestras: 10
edge-processor | [Edge] Procesadas 10 lecturas. Latencia prom: 1093ms. Enviando...
edge-processor | [Edge] Fallo al contactar al coordinador (intento 5): error sending request for url (http://10.10.10.1:
3000/submit_report): error trying to connect: tcp connect error: Connection refused (os error 111)
edge-processor | [Edge] No se pudo enviar el reporte después de 5 intentos
edge-processor | [Edge] Reintentando en 100ms...
edge-processor | [Edge] Fallo al contactar al coordinador (intento 1): error sending request for url (http://10.10.10.1:
3000/submit_report): error trying to connect: tcp connect error: Connection refused (os error 111)
edge-processor | [Edge] ❤️Heartbeat fallido: error sending request for url (http://10.10.10.1:3000/heartbeat): error try
ing to connect: tcp connect error: Connection refused (os error 111)
edge-processor | [Edge] Fallo al contactar al coordinador (intento 2): error sending request for url (http://10.10.10.1:
3000/submit_report): error trying to connect: tcp connect error: Connection refused (os error 111)
edge-processor | [Edge] Reintentando en 200ms...
edge-processor | [Edge] Fallo al contactar al coordinador (intento 3): error sending request for url (http://10.10.10.1:
3000/submit_report): error trying to connect: tcp connect error: Connection refused (os error 111)
edge-processor | [Edge] Reintentando en 400ms...
edge-processor | [Edge] Fallo al contactar al coordinador (intento 4): error sending request for url (http://10.10.10.1:
3000/submit_report): error trying to connect: tcp connect error: Connection refused (os error 111)
edge-processor | [Edge] Reintentando en 800ms...
edge-processor | [Edge] Fallo al contactar al coordinador (intento 5): error sending request for url (http://10.10.10.1:
3000/submit_report): error trying to connect: tcp connect error: Connection refused (os error 111)
edge-processor | [Edge] No se pudo enviar el reporte después de 5 intentos
```

Pruebas de Red con tc netem

La demostración validará mediante `iperf3` (o logs de procesamiento) el comportamiento del pipeline en la recepción de datos bajo condiciones de red adversas controladas.

Pruebas de Red sin tc netem

```
gabolectric@ubuntu-dev:~/Pipeline_IoT-Edge$ iperf3 -u -c 10.10.10.1 -b 10M -i 0.5 -p 5201
Connecting to host 10.10.10.1, port 5201
[  5] local 10.10.10.2 port 37341 connected to 10.10.10.1 port 5201
[ ID] Interval           Transfer     Bitrate         Total Datagrams
[  5]   0.00-0.50   sec   611 KBytes  10.0 Mbits/sec  457
[  5]   0.50-1.00   sec   609 KBytes  9.98 Mbits/sec  456
[  5]   1.00-1.50   sec   611 KBytes  10.0 Mbits/sec  457
[  5]   1.50-2.00   sec   611 KBytes  10.0 Mbits/sec  457
[  5]   2.00-2.50   sec   611 KBytes  10.0 Mbits/sec  457
[  5]   2.50-3.00   sec   611 KBytes  10.0 Mbits/sec  457
[  5]   3.00-3.50   sec   611 KBytes  10.0 Mbits/sec  457
[  5]   3.50-4.00   sec   611 KBytes  10.0 Mbits/sec  457
[  5]   4.00-4.50   sec   609 KBytes  9.98 Mbits/sec  456
[  5]   4.50-5.00   sec   611 KBytes  10.0 Mbits/sec  457
[  5]   5.00-5.50   sec   611 KBytes  10.0 Mbits/sec  457
[  5]   5.50-6.00   sec   611 KBytes  10.0 Mbits/sec  457
[  5]   6.00-6.50   sec   611 KBytes  10.0 Mbits/sec  457
[  5]   6.50-7.00   sec   611 KBytes  10.0 Mbits/sec  457
[  5]   7.00-7.50   sec   611 KBytes  10.0 Mbits/sec  457
[  5]   7.50-8.00   sec   611 KBytes  10.0 Mbits/sec  457
[  5]   8.00-8.50   sec   609 KBytes  9.98 Mbits/sec  456
[  5]   8.50-9.00   sec   611 KBytes  10.0 Mbits/sec  457
[  5]   9.00-9.50   sec   611 KBytes  10.0 Mbits/sec  457
[  5]   9.50-10.00  sec   611 KBytes  10.0 Mbits/sec  457
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval           Transfer     Bitrate         Jitter    Lost/Total Datagrams
[  5]   0.00-10.00  sec  11.9 MBytes  10.0 Mbits/sec  0.000 ms  0/9137 (0%)  sender
[  5]   0.00-10.04  sec  11.9 MBytes  9.96 Mbits/sec  1.126 ms  0/9136 (0%)  receiver

iperf Done.
gabolectric@ubuntu-dev:~/Pipeline_IoT-Edge$
```

```
[rogeliovm@archlinux Pipeline_IoT-Edge]$ iperf3 -s -p 5201
-----------------------------------------------------------
Server listening on 5201 (test #1)
-----------------------------------------------------------
        Accepted connection from 10.10.10.2, port 39000
[  5] local 10.10.10.1 port 5201 connected to 10.10.10.2 port 37341
[ ID] Interval           Transfer     Bitrate         Jitter    Lost/Total Datagrams
[  5]   0.00-1.00   sec  1.16 MBytes  9.73 Mbits/sec  1.054 ms  0/890 (0%)
[  5]   1.00-2.00   sec  1.19 MBytes  9.99 Mbits/sec  1.205 ms  0/913 (0%)
[  5]   2.00-3.00   sec  1.19 MBytes  9.98 Mbits/sec  1.162 ms  0/912 (0%)
[  5]   3.00-4.00   sec  1.19 MBytes  10.0 Mbits/sec  1.114 ms  0/914 (0%)
[  5]   4.00-5.00   sec  1.19 MBytes  10.0 Mbits/sec  1.030 ms  0/914 (0%)
[  5]   5.00-6.00   sec  1.19 MBytes  9.98 Mbits/sec  1.143 ms  0/912 (0%)
[  5]   6.00-7.00   sec  1.19 MBytes  10.0 Mbits/sec  1.259 ms  0/915 (0%)
[  5]   7.00-8.00   sec  1.19 MBytes  10.0 Mbits/sec  1.108 ms  0/914 (0%)
[  5]   8.00-9.00   sec  1.19 MBytes  10.0 Mbits/sec  1.082 ms  0/915 (0%)
[  5]   9.00-10.00  sec  1.19 MBytes  9.96 Mbits/sec  1.088 ms  0/911 (0%)
[  5]  10.00-10.04  sec  34.7 KBytes  7.87 Mbits/sec  1.126 ms  0/26 (0%)
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval           Transfer     Bitrate         Jitter    Lost/Total Datagrams
[  5]   0.00-10.04  sec  11.9 MBytes  9.96 Mbits/sec  1.126 ms  0/9136 (0%)  receiver
-----------------------------------------------------------
Server listening on 5201 (test #2)
-----------------------------------------------------------
```

Escenario base sin degradación artificial para contrastar la latencia y el rendimiento nominal del sistema.

Pruebas de Red con tc netem

```
iperf3: error - the server has terminated
gabolectric@ubuntu-dev:~/Pipeline_IoT-Edge$ iperf3 -u -c 10.10.10.1 -b 10M -i 0.5 -p 5201
Connecting to host 10.10.10.1, port 5201
[  5] local 10.10.10.2 port 42175 connected to 10.10.10.1 port 5201
[ ID] Interval           Transfer     Bitrate         Total Datagrams
[  5]   0.00-0.50   sec   611 KBytes  10.0 Mbits/sec  457
[  5]   0.50-1.00   sec   609 KBytes  9.98 Mbits/sec  456
[  5]   1.00-1.50   sec   611 KBytes  10.0 Mbits/sec  457
[  5]   1.50-2.00   sec   611 KBytes  10.0 Mbits/sec  457
[  5]   2.00-2.50   sec   611 KBytes  10.0 Mbits/sec  457
[  5]   2.50-3.00   sec   611 KBytes  10.0 Mbits/sec  457
[  5]   3.00-3.50   sec   611 KBytes  10.0 Mbits/sec  457
[  5]   3.50-4.00   sec   611 KBytes  10.0 Mbits/sec  457
[  5]   4.00-4.50   sec   609 KBytes  9.98 Mbits/sec  456
[  5]   4.50-5.00   sec   611 KBytes  10.0 Mbits/sec  457
[  5]   5.00-5.50   sec   611 KBytes  10.0 Mbits/sec  457
[  5]   5.50-6.00   sec   611 KBytes  10.0 Mbits/sec  457
[  5]   6.00-6.50   sec   611 KBytes  10.0 Mbits/sec  457
[  5]   6.50-7.00   sec   611 KBytes  10.0 Mbits/sec  457
[  5]   7.00-7.50   sec   611 KBytes  10.0 Mbits/sec  457
[  5]   7.50-8.00   sec   611 KBytes  10.0 Mbits/sec  457
[  5]   8.00-8.50   sec   609 KBytes  9.98 Mbits/sec  456
[  5]   8.50-9.00   sec   611 KBytes  10.0 Mbits/sec  457
[  5]   9.00-9.50   sec   611 KBytes  10.0 Mbits/sec  457
[  5]   9.50-10.00  sec   611 KBytes  10.0 Mbits/sec  457
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval           Transfer     Bitrate         Jitter    Lost/Total Datagrams
[  5]   0.00-10.00  sec  11.9 MBytes  10.0 Mbits/sec  0.000 ms  0/9137 (0%)  sender
[  5]   0.00-22.20  sec  4.76 MBytes  1.80 Mbits/sec  119.947 ms  5484/9135 (60%)  receiver

iperf Done.
gabolectric@ubuntu-dev:~/Pipeline_IoT-Edge$
```

```
[rogeliovm@archlinux Pipeline_IoT-Edge]$ iperf3 -s -p 5201
-----------------------------------------------------------
Server listening on 5201 (test #1)
-----------------------------------------------------------
Accepted connection from 10.10.10.2, port 39406
[  5] local 10.10.10.1 port 5201 connected to 10.10.10.2 port 42175
[ ID] Interval           Transfer     Bitrate         Jitter    Lost/Total Datagrams
[  5]   0.00-1.00   sec  0.00 Bytes  0.00 bits/sec  0.000 ms  0/0 (0%)
[  5]   1.00-2.00   sec  0.00 Bytes  0.00 bits/sec  0.000 ms  0/0 (0%)
[  5]   2.00-3.00   sec   216 KBytes  1.77 Mbits/sec  301.715 ms  661/823 (80%)
[  5]   3.00-4.00   sec   725 KBytes  5.94 Mbits/sec  483.777 ms  25/568 (4.4%)
[  5]   4.00-5.00   sec   409 KBytes  3.35 Mbits/sec  856.470 ms  1008/1314 (77%)
[  5]   5.00-6.00   sec   290 KBytes  2.38 Mbits/sec  590.675 ms  608/825 (74%)
[  5]   6.00-7.00   sec   586 KBytes  4.80 Mbits/sec  575.424 ms  483/922 (52%)
[  5]   7.00-8.00   sec   448 KBytes  3.67 Mbits/sec  686.873 ms  530/865 (61%)
[  5]   8.00-9.00   sec   331 KBytes  2.71 Mbits/sec  616.467 ms  738/986 (75%)
[  5]   9.00-10.00  sec   517 KBytes  4.24 Mbits/sec  545.779 ms  491/878 (56%)
[  5]  10.00-11.00  sec   441 KBytes  3.61 Mbits/sec  563.061 ms  555/885 (63%)
[  5]  11.00-12.00  sec   402 KBytes  3.29 Mbits/sec  452.225 ms  652/953 (68%)
[  5]  12.00-13.00  sec   374 KBytes  3.06 Mbits/sec  391.357 ms  -164/116 (-1.4e+02%)
[  5]  13.00-14.00  sec   138 KBytes  1.13 Mbits/sec  119.947 ms  -103/0 (0%)
[  5]  14.00-15.00  sec  0.00 Bytes  0.00 bits/sec  119.947 ms  0/0 (0%)
[  5]  15.00-16.00  sec  0.00 Bytes  0.00 bits/sec  119.947 ms  0/0 (0%)
[  5]  16.00-17.00  sec  0.00 Bytes  0.00 bits/sec  119.947 ms  0/0 (0%)
[  5]  17.00-18.00  sec  0.00 Bytes  0.00 bits/sec  119.947 ms  0/0 (0%)
[  5]  18.00-19.00  sec  0.00 Bytes  0.00 bits/sec  119.947 ms  0/0 (0%)
[  5]  19.00-20.00  sec  0.00 Bytes  0.00 bits/sec  119.947 ms  0/0 (0%)
[  5]  20.00-21.00  sec  0.00 Bytes  0.00 bits/sec  119.947 ms  0/0 (0%)
[  5]  21.00-22.00  sec  0.00 Bytes  0.00 bits/sec  119.947 ms  0/0 (0%)
[  5]  22.00-22.20  sec  0.00 Bytes  0.00 bits/sec  119.947 ms  0/0 (0%)
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval           Transfer     Bitrate         Jitter    Lost/Total Datagrams
[SUM]  0.0-22.2 sec  3505 datagrams received out-of-order
[  5]   0.00-22.20  sec  4.76 MBytes  1.80 Mbits/sec  119.947 ms  5484/9135 (60%)  receiver
-----------------------------------------------------------
Server listening on 5201 (test #2)
-----------------------------------------------------------
```

Pruebas de Red con tc netem

latencia_vpn.sh

netem > latencia_vpn.sh

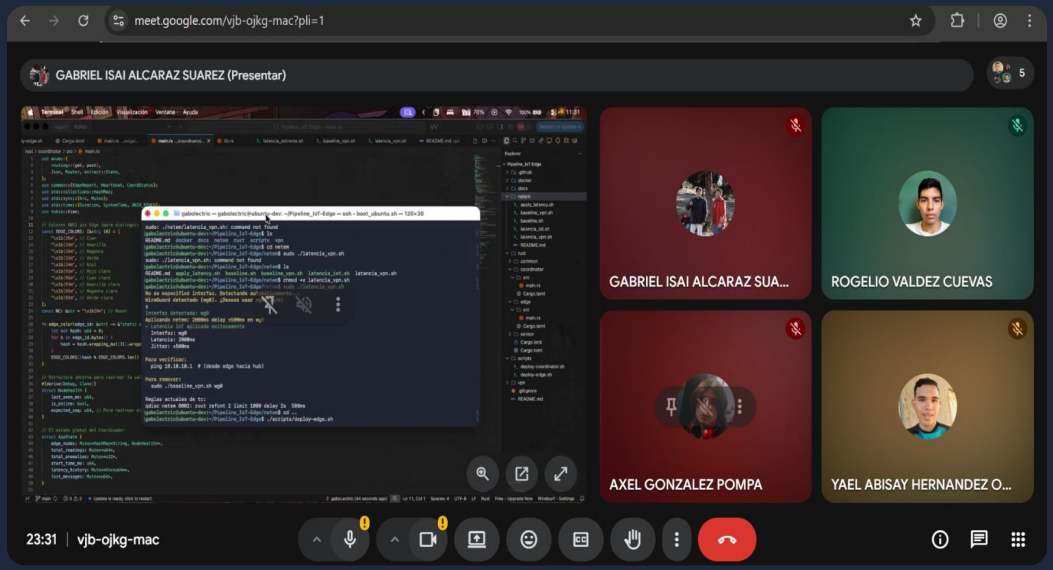
```
#!/bin/bash
# Escenario Latencia IoT - delay 3000ms jitter 1000ms 25% (Documento B)
# Uso: sudo ./latencia_vpn.sh [interfaz]
    ⌘L for Agent
YELLOW='\033[1;33m'
NC='\033[0m'

INTERFACE=${1:-wg0}

if ! ip link show "$INTERFACE" &> /dev/null; then
    echo "Error: Interfaz '$INTERFACE' no existe"
    exit 1
fi

tc qdisc del dev "$INTERFACE" root 2>/dev/null || true
tc qdisc add dev "$INTERFACE" root netem delay 3000ms 1000ms 25%

echo -e "${YELLOW}✓ Latencia 3000ms ±1000ms (25% corr) aplicada en $INTERFACE${NC}"
echo "  ping 10.10.10.1  # Verificar"

```

perdida_paquetes.sh

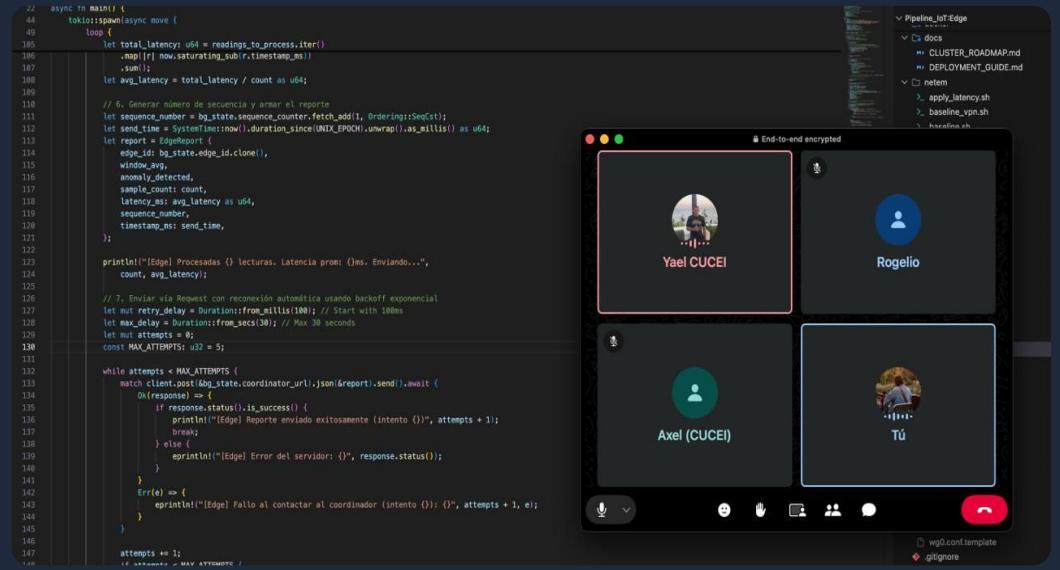
netem > perdida_paquetes.sh

```
#!/bin/bash
# Pérdida de Paquetes - loss 8% (Documento B). Uso: sudo ./perdida_paquetes.sh [interfaz]

RED='\033[0;31m'
NC='\033[0m'

INTERFACE=${1:-wg0}

if ! ip link show "$INTERFACE" &> /dev/null; then
    INTERFACE=$(ip route | grep default | head -n1 | awk '{print $5}')
fi

tc qdisc del dev "$INTERFACE" root 2>/dev/null || true    Tab to Jump
tc qdisc add dev "$INTERFACE" root netem loss 8%

echo -e "${RED}✓ Pérdida 8% aplicada en $INTERFACE${NC}"
echo "  ping -c 10 10.10.10.1  # Verificar (~8% loss)"

```

Evidencia de Reuniones y Validación Ágil



Reunión Sprint 1 (Sincronización): Confirmar la arquitectura del sistema de tres nodos lógicos



Reunión Sprint 1 (Técnica): Definición de reglas tc netem y arquitectura del monitor de red.Validación de Edge y Configuración de Red

Estas evidencias validan el trabajo colaborativo requerido para el Proyecto 2 en el Sprint 1, asegurando la trazabilidad de las decisiones técnicas y la asignación equitativa de tareas.

Retos Encontrados

Sincronización Docker / VPN

Asegurar que los contenedores enruten correctamente el tráfico interno a través de la interfaz wg0 del host sin comprometer su aislamiento de red predeterminado.

Flujo Asíncrono en Rust

Implementación concurrente utilizando el framework Tokio para evitar el bloqueo (race conditions) durante la recepción y reenvío continuo de telemetría.

Aislamiento de Reglas tc

Aplicar correctamente Traffic Control (tc netem) de modo que afecte el tráfico de extremo a extremo, pero se mida correctamente antes de la desencapsulación.

Retos Encontrados

```
gabolectric@ubuntu-dev:~/Pipeline_IoT-Edge$ ./netem/latencia_vpn.sh
RTNETLINK answers: Operation not permitted
✔ Latencia 80ms ±20ms aplicada en wg0
  ping 10.10.10.1  # Verificar
```

```
gabolectric@ubuntu-dev:~/Pipeline_IoT-Edge$ ./netem/latencia_vpn.sh
RTNETLINK answers: Operation not permitted
✔ Latencia 3000ms ±1000ms (25% corr) aplicada en wg0
  ping 10.10.10.1  # Verificar
```

¿Preguntas?

Paso a la demostración técnica en vivo en la terminal.